1. A teammate accidentally force-pushed to main and rewrote history. How do you recover and stabilize the repo?

A: You recover using the **git reflog** command. This command shows a log of every action (like a journal) that happened in your local repository. You find the commit ID of the correct, stable state before the force-push. Then, you use **git reset --hard <correct_commit_id>** to point the main branch back to the good state, stabilizing the repository.

2. A developer pushed sensitive data (passwords/API keys) into Git history. How do you remove it safely from ALL commits?

A: This requires rewriting the entire history using a specialized tool like the **git filter-repo** utility (or the older git filter-branch). You specify the file or content you want to remove. The tool then walks through every commit in the history, deletes the sensitive data, and creates new commit IDs. Finally, everyone must **force-push** the cleaned history to the remote.

3. Two developers merged huge conflicting changes and now the branch is unstable. How do you resolve the conflict and prevent this happening again?

A: To resolve the current conflict, you must manually edit the files, combine the code correctly, and complete the merge commit. To prevent this next time, enforce a policy of **frequent small commits** and **rebase/merge often** from the base branch. Also, require **code reviews** for large changes and run automated tests before any merge is allowed.

4. A teammate created a feature on the wrong base branch. How do you move the entire feature to the correct branch without losing work?

A: You use **git rebase --onto** to detach the commits and attach them to a new base.

1. First, create the new correct branch.
2. Then, run git rebase --onto <correct_base> <wrong_base> <feature_branch>. This command copies the feature's commits and reapplies them cleanly on top of the correct base branch's history.

5. Production broke due to a bad commit. The last 5 commits are mixed with other features. How do you identify the faulty commit and roll it back without losing unrelated work?

A: You use **git bisect** to quickly find the faulty commit by testing versions between "good" and "bad" commits. Once the bad commit is identified, you use **git revert <faulty_commit_id>**. The revert command creates a **new commit** that undoes only the specific changes of the faulty commit, leaving the unrelated, good commits untouched.